

REPORT
on
NOISE MONITORING SYSTEM USING ESP32

Submitted by

Yug Patel(IU 2341230117)
Vraj Mehta (IU 2341230136)
Keval Sheth (IU 2341230094)

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING



INSTITUTE OF TECHNOLOGY AND ENGINEERING
INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA,

WEB: www.indusuni.ac.in

Abstract

This project presents the practical implementation of a Noise Monitoring System using an ESP32 microcontroller, a sound sensor module, and three LEDs. The system monitors environmental sound levels in real time and classifies them into categories such as quiet, moderate, loud, and very loud. Data is also transmitted wirelessly using Bluetooth Low Energy (BLE) and can be viewed on a mobile application developed using Flutter.

The project is implemented using Arduino IDE and a CH340 USB driver for communication between the ESP32 and the computer. The design focuses on simplicity, real-world usability, and student-level understanding.

1. Introduction

Noise pollution is a growing concern in modern environments. This project provides a simple and effective way to monitor sound levels and visualize them using LEDs as well as a mobile application.

The system not only shows local indication through LEDs but also sends live data to a mobile app using BLE, making it more practical and interactive.

2. Objectives

- Design a real-time noise monitoring system
- Use ESP32 with Arduino IDE
- Provide LED-based indication
- Transmit data via BLE
- Develop a mobile app using Flutter

3. Components Used

- ESP32 Development Board (CH340 based)
- Sound Sensor Module
- Breadboard
- Jumper Wires
- 3 LEDs (Green, Yellow, Red)
- 220Ω Resistors
- USB Cable

Software:

- Arduino IDE

- CH340 Driver
- Flutter (for mobile app)

4. System Architecture

Input

Sound sensor captures environmental noise and converts it to analog signals.

Processing

ESP32 reads the analog value and processes it using programmed thresholds.

Output

- LEDs show noise level visually
- BLE sends data to mobile app

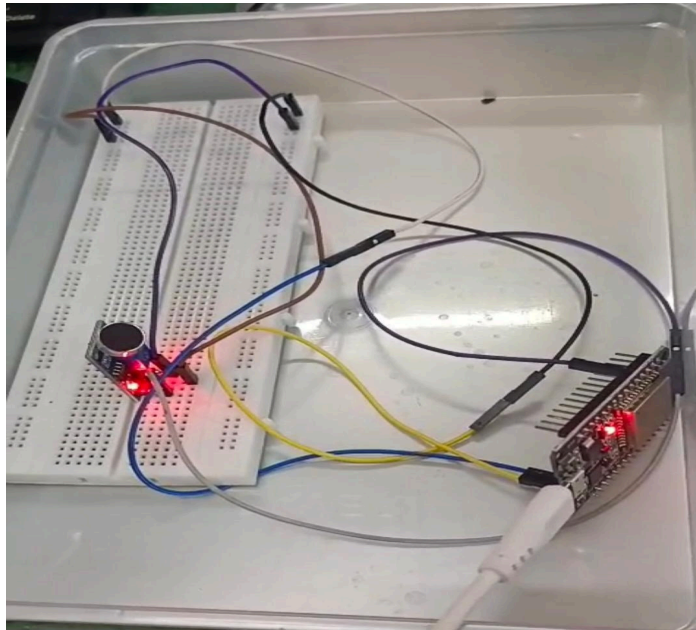
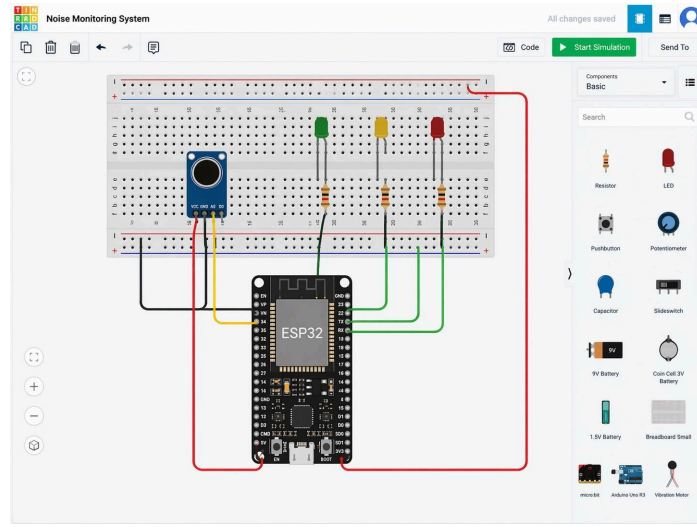
5. Circuit Description

Sound Sensor

- VCC → 3.3V
- GND → GND
- A0 → GPIO34

LEDs

- Green → GPIO23
- Yellow → GPIO22
- Red → GPIO21
- Each LED connected with 220Ω resistor



6. Working Principle

- Sensor detects sound
- ESP32 reads analog value
- Value compared with thresholds
- LED is activated
- Data sent via BLE

7. Implementation Logic

The system works in two parallel ways:

1. Hardware Logic

- Read sound value
- Compare thresholds
- Turn ON corresponding LED

2. Wireless Communication

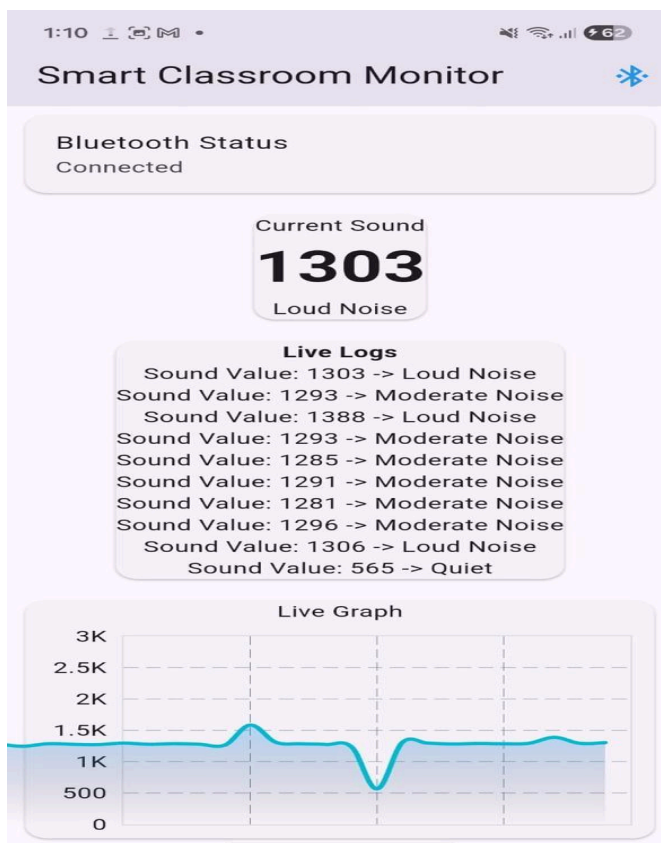
- ESP32 creates BLE server
- Mobile app connects to ESP32.Noise data is sent continuously
- App displays real-time values

8. Mobile Application (Flutter)

A simple Flutter application is developed to receive BLE data from ESP32.

Features:

- Scan and connect to ESP32 device
- Display sound value
- Show noise level status
- Real-time updates



```
-----
Sound Value: 1375 -> Moderate Noise
Sound Value: 1369 -> Moderate Noise
Sound Value: 1373 -> Moderate Noise
Sound Value: 1383 -> Moderate Noise
Sound Value: 1413 -> Loud Noise
Sound Value: 1355 -> Moderate Noise
Sound Value: 1375 -> Moderate Noise
```

9. Arduino Code

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#define SOUND_PIN 34
#define SERVICE_UUID "12345678-1234-1234-1234-1234567890ab"
#define CHARACTERISTIC_UUID "abcd1234-5678-1234-5678-abcdef123456"
```

```
BLECharacteristic *pCharacteristic;
bool deviceConnected = false;
```

```
int quietLevel = 800;
int moderateLevel = 1300;
int loudLevel = 2000;
```

```
class MyCallbacks: public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) {
    deviceConnected = true;
    Serial.println("Device Connected");
  }
  void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    Serial.println("Device Disconnected");
    BLEDevice::startAdvertising();
  }
};
```

```
void setup() {
  Serial.begin(115200);

  BLEDevice::init("ESP32_Noise");
```

```
BLEServer *pServer =
BLEDevice::createServer();
pServer->setCallbacks(new MyCallbacks());
BLEService *pService = pServer->createService(SERVICE_UUID);

pCharacteristic = pService->createCharacteristic(

    CHARACTERISTIC_UUID,
    BLECharacteristic::PROPERTY_NOTIFY | BLECharacteristic::PROPERTY_READ
);

BLE2902 *desc = new BLE2902();
desc->setNotifications(true);
pCharacteristic->addDescriptor(desc);
pCharacteristic->setValue("Waiting...");

pService->start();

BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
pAdvertising->setScanResponse(true);
pAdvertising->start();

Serial.println("BLE Ready");
}

void loop() {

    int soundValue = analogRead(SOUND_PIN);

    String status = "";

    if (soundValue < quietLevel) {
        status = "Quiet";
    }
    else if (soundValue < moderateLevel) {
        status = "Moderate Noise";
    }
    else if (soundValue < loudLevel) {
        status = "Loud Noise";
    }
    else {
        status = "Very Loud";
    }

    String logMessage = "Sound Value: " + String(soundValue) + " -> " + status;
```

```
Serial.println(logMessage);
```

```
if (deviceConnected) {  
    pCharacteristic->setValue(logMessage.c_str());  
    pCharacteristic->notify();  
    delay(50);  
}  
  
delay(800);  
}
```

10. Advantages

- Simple and practical
- Real-time monitoring
- Wireless data transmission
- Mobile app integration

11. Applications

- Schools
- Hospitals
- Offices
- Smart environments

12. Conclusion

This project successfully demonstrates a real-time noise monitoring system using ESP32, BLE communication, and a Flutter mobile application. It is a practical and student-friendly implementation that combines hardware and software effectively.